TEMCO LEARN

Temco Platform

The platform · built Nov 10, 2026

The platform

The center documentation platform. One internal application — VS Code plus this extension — that takes documentation from **many kinds of source**, lets people manage and edit it, and **builds the final documents**: a site, a CHM, a PDF. A local AI reads the history of those sources and drafts the updates. Everything it knows lives in a local **SQLite catalog**.

This document is the platform's own design. The **content** rules — the registry schema, the checks, the review flow, the publishing engine — are the design package in `design/temco-learn/`, which stays the source of truth.

How Microsoft's own documentation is managed, for comparison

Worth stating, because it is the shape being copied:

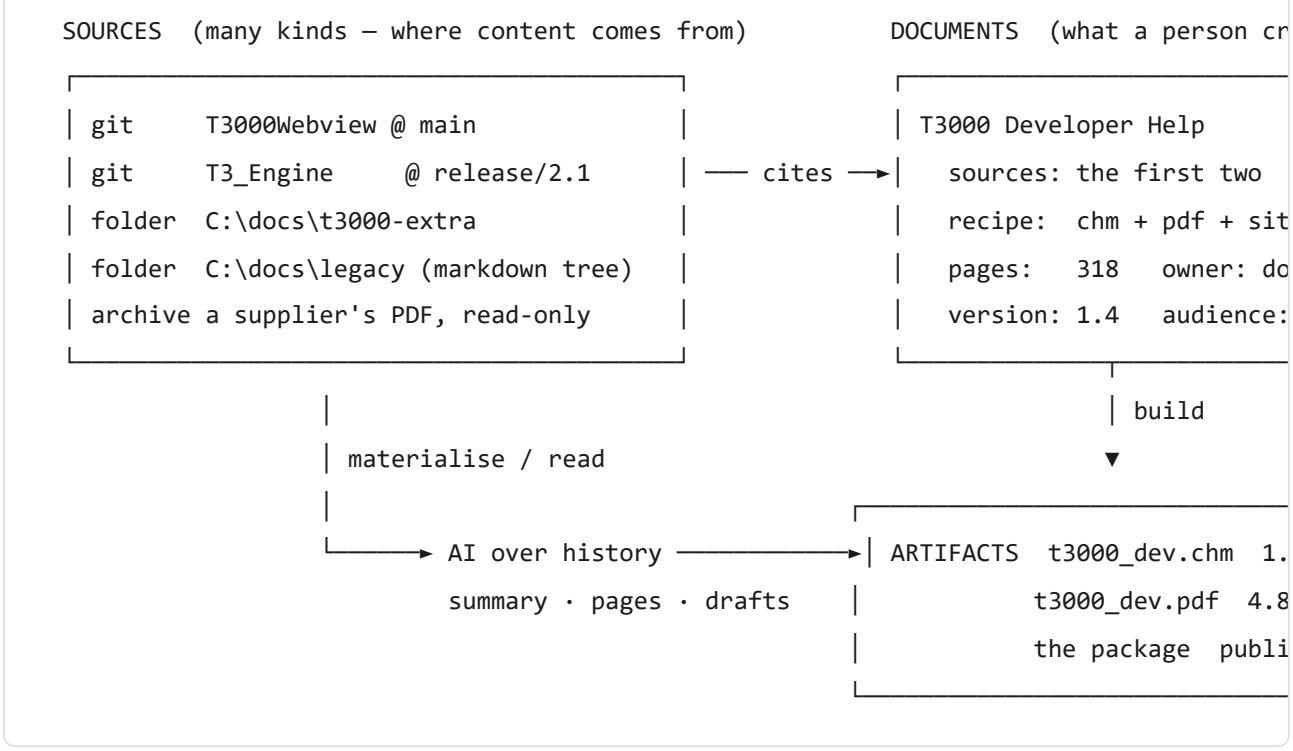
- **A docset** is a set of Markdown pages plus a table of contents, living in a repository (Microsoft's own docs are a private repo of Markdown).
- **Metadata** on each page — `author`, `ms.topic`, `ms.date` — decides where it appears and who owns it.
- **Monikers** are version markers: one page can serve several product versions, and the build emits the right one per version.
- **A build pipeline** turns the docset into the published site, and separately into an offline PDF.
- **The reader** gets one searchable site; the *authoring* side (review, status, ownership) is internal.

The equivalents: docset ≈ a **Document**, moniker ≈ its **version**, pipeline ≈ its **recipe**. The difference is where it runs: theirs is a hosted service, ours is an internal app on the maintainer's machine — and ours must also produce the **CHM**, which theirs does not.

The model: sources → documents → artifacts

The unit of work is **not a repository**. A repository is where content may *start*; the thing a person creates, owns, versions and builds is a **document** — "T3000 Developer Help", shipped as `t3000_dev.chm`; "T3000Webview Guide", shipped as `t3000_webview.pdf`.

TEXT



Source — where content comes from. Not necessarily GitHub, not necessarily a repository, not necessarily Markdown:

Kind	Examples	What the platform does
`git`	any remote: GitHub, GitLab, a bare path — `owner/repo@branch`	reads the tree, the history and the diffs
`folder`	a docfx project, a Markdown tree, loose HTML, images	reads it in place; never moves it
`archive`	a supplier's PDF, a vendor's ZIP of pages	inventories it, read-only, for reference and search

The **branch is chosen per source**, defaulting to the repository's own default (`main`), and reported when the documentation is not on it.

Document — what a person creates: a name, its sources and their branches, a **recipe** (how to build it), a **version**, an audience, an owner, and the outputs it must produce. This is the platform's unit of management, and it is what the generator, the builds, the health and the AI are all *about*.

Artifact — a file a build produced: path, size, time, hash, and which document and source revision it came from.

Build — one run of a document's recipe at a source revision: a job with a log, a status, and the artifacts it left behind. Success is an artifact that did not exist before, never an exit code.

Analysis — what the local AI did over the history: a summary of a commit, the pages it affects, a draft, and what is still owed.

The catalog

The platform's memory: documents, sources, builds, artifacts, commits and what changed since you last looked. Relational, long-lived, queryable offline.

Where: `context.globalStorageUri` of the **store module** — a `catalog.db` and a `sources/` cache of materialised trees. One window, one file, opened by the module that owns it: a caller that opened it with its own storage would find an empty database beside its own code. **Never inside a repository**, never committed, never in a product's git history.

```SQL
sources     id · kind(git|folder|archive) · name · remote_url · path · default_branch ·
            last_fetch_at · last_commit_sha · last_commit_at · size_bytes · state

documents   id · name · slug · version · audience · owner · recipe(kind) · created_at ·

doc_sources document_id · source_id · include_glob · branch · role(primary|supplement)

recipes     document_id · engine(docfx|own-script|chm|pdf|site) · config_path · script ·

builds      id · document_id · started_at · finished_at · status · revision · log_path

artifacts   id · build_id · document_id · path · bytes · built_at · sha256 · kind(chm|pd

commits     sha · source_id · at · author · subject · summary · summary_model · summaris

commit_pages  commit_sha · page_id · how(code-glob|path|terms)

pages       id · document_id · source_id · path · title · owner · reviewed_at · stale ·

scans       id · source_id · at · mode(local|fetch|api) · revision · counts_json

events      id · at · kind · subject_kind · subject_id · detail · seen_at
            kind ∈ { new_source, docs_found, registry_found, push, pages_changed,
                     docs_removed, branch_mismatch, build_ok, build_failed, draft_ready
```

Every view is a query over these rows. Nothing is asked of the network twice, and the last scan is readable with the network unplugged — and says when it was taken.

Reading a source: three ways, and when to use which

The user's requirement — "*load the source code… download to a local temp folder or read it from online*" — is two mechanisms with different costs, and the platform picks per job:

Mode	How	Used for	Cost
local	a folder you have open, or a path in the catalog	everything: files, checks, edits, builds	none
materialised	a shallow fetch into `globalStorage/sources/<slug>` — `git clone --filter=blob:none --depth 1`, or a fetch of the needed range — on a scratch branch the platform owns	drafts (the model needs the page text), builds, file-existence checks	disk, bounded
online	the host's API for one commit's changed files and patches	commit summaries, "what changed" — *diffs* only, no clone	API calls, bounded

The rule that picks between the last two: **summaries need diffs; drafts need files.** A commit summary over an API patch is cheap and needs no clone; a draft needs the real page, so the platform materialises the source once and reuses it. A patch can be truncated for a very large file — the summary records that it was, rather than guessing.

Caps, because this is a maintainer's PC: a source cache budget (default 2 GB) with LRU eviction, a maximum fetch depth, a maximum commits analysed per run, and a *Prune the source cache* command. Nothing is ever fetched without a command or a queue the user started.

The AI over history, per source

TEXT

```
commits since the last scan   ──►  summarise what it did, in the product's words
    (API patches, or the local clone)      │
                                           ├──► which pages it affects
                                           │      code globs from the registry, else
                                           ├──► draft the page edits  (local model, d
                                           └──► add to the owed list
▲  nothing here publishes, commits or approves: a named person does that
```

Summaries, their model and their time are stored, so a summary is computed once and is visible offline. The whole pass is a **job with progress** — resumable, bounded per run, and honest about what it skipped.

Creating a document

Create a document asks for the few facts the catalog needs, and nothing more:

```text
   name        T3000 Developer Help
   sources     [x] T3000Webview  branch main        [x] C:\docs\t3000-extra
               [ ] T3_Engine     branch release/2.1
   recipe      ( ) site   ( ) pdf   (x) chm   [x] also the site
   version     1.4            audience  developers            owner  docs
   ──────────────────────────────────────────────────────────────────────

   the platform answers with what it can see:  318 pages · 5 sources · 2 sources carry a
                                                suggested: run the T3000Webview generato
```

A document can be created before its sources are perfect — that is the point. The platform records what it found, says what is missing, and offers the next step.

Editing and writing

- **A local source is edited in place** — the page file is the page, as the Content view already does.
- **A git source is edited on a draft branch** in the materialised clone: the AI's diff is applied there, the user reviews it, and *they* commit or open a PR. The platform never commits on anyone's behalf.
- **Nothing under `_admin/` or `_edit/` is ever written** — `00-architecture.md` calls those *generated pages, not documentation.*
- The hosted editor stays the design package's web surface; this is its desktop equivalent.

Telling the user — quietly

Unseen events become **one** grouped notification, never one per event, and `seen_at` is stamped when the group is shown or dismissed so a fact is never announced twice. Default `docs-only`: a daily summary of documentation change, no modals. A *Since you last looked* node heads the view.

Feasibility

Yes. It is a catalog, a few connectors, a build runner and the AI that already exists — no new engine, and no server. The real constraints, each with its answer:

Constraint	Answer
The database — decided `node:sqlite`: it ships with the extension host's Node, so there is no dependency to add	a build whose Node predates 22.5 is refused in words by the store, at the point of use. The `engines.vscode` floor should be raised once the first F5 confirms the built-in is present
Disk — cached source trees grow	a budget (2 GB default), LRU eviction, *Prune the source cache*, and a `.gitignore`-aware size report
The network — a whole-organisation read and commit patches need rate limit	a token in secret storage (5,000 calls/hour), and *online* mode on demand only. The 60-call limit has been reached in practice; it is not an optional convenience
Time — AI summaries over history are slow	a job queue with progress, a per-run cap, resumable, and summaries stored once
Many kinds of local documentation	one ingester per kind, starting with: a Markdown tree, a docfx project, and an inventory-only reader for HTML and PDF
Trust — this is the maintainer's machine	all state in `globalStorageUri`; nothing written into a repository; nothing fetched without a command or a queue the user started

Surfaces

The platform has **two lists** in the sidebar and **rich pages** in the editor area. Nothing else competes for attention: a document is managed from its page, a source is read from its row, and the AI is beside whatever you are working on.

What opens when the platform is activated is the project you are on — its dashboard, the one surface that says what is in it and what next — then the page you were last working on, then the home page when there is neither. `temcoWorkbench.start.open` (`project` · `page` · `home` · `welcome` · `none`) chooses, and `none` opens no tab at all for anyone who would rather nothing opened by itself. The side panel the window is revealed on is a second, separate choice — `temcoWorkbench.start.panel`, whose values are the left panel's own items (Home, Documentation, Repositories, AI, Forum, Tools, Settings) and whose default is `temcoDashboard`.

The Workspace is the first view, and the Library is beneath it. Both sit in the `temco` activity bar: the Workspace tree is the documentation module's, the Library is the workbench's — the one index of everything every module has. The Workspace shows the project being worked on and never the open folders, so the registry checks, the status bar and the faults follow the project too — a tree built from whatever happens to be open would be a second, unrelated answer to the same question. The view is never hidden and never blank: with nothing to show it carries **rows that say why and offer the one act that fills it** (*no project yet* → create one; *no project is being worked on* → choose one in the Library; *this project has nothing to show* → open it and read its sources). Those rows are drawn by the tree itself —

`viewsWelcome` was tried first and dropped, because a welcome's `when` clause is resolved when the view is first painted, which is before the refresh has read anything: it announced "nothing recorded yet" over a catalog that held five projects.

With exactly one project recorded there is nothing to choose between, so it selects itself and the tree is ready without anyone picking it; that is not a person choosing, so the sidebar is left where it was. Choosing a project afterwards brings the Workspace forward, and clearing one brings the Library back — the two views are a sequence, and only a click walks it.

Nothing here is specific to a product. The names in the figures below are examples; the code knows no repository, no branch and no product name, and it reads whatever the catalog and the editor are told about.

TEXT

```
SIDEBAR (activity bar — the two lists)      EDITOR AREA (the pages)          RIGHT SID
┌──────────────┐                            ┌──────────────────────┐         ┌────────
│ DOCUMENTS    │  what you manage           │ Document             │         │ TEMCO A
│ SOURCES      │  what you read from        │ Create a document    │         │  the AI
└──────────────┘                            │ AI analysis (job)    │         │  what y
                                            │ Build log            │         └────────
                                            └──────────────────────┘
```

The project flow

The flow was reworked after the first attempt at it was judged — correctly — not good: a project's pages existed in two places with two truths, a source could not be acted on from the row that named it, and *Add a source* threw the whole edit form at you. What changed, in the order it matters:

TEXT

```
selected project ──────────► the Workspace view, the counts, the faults, the AI
      │                        (one world: the view follows the project, not the open f
      ├── Library row  ⚙ Read · Open the source · Set the branch · Forget     (action
      ├── hub          Read every source · Add a source · Build · Edit        (the wor
      └── hub ▸ Sources  the add form lives here — a source is one decision, not the w
```

#	What it was	What it is
A	the Workspace view showed the open folders; a project's pages lived in the catalog, beside it	**one world**: with a project being worked on, the view shows *that project's* sources — a local folder gets the full checks even when the window has it closed, and a source that is not here shows what the read recorded, marked `origin: 'catalog'` so nothing unchecked looks checked. With no project it says why instead of showing the folders
B	a Library row was text; reading was only on the view's title bar	rows carry a context value — **Read · Open the source · Set the branch · Forget** on a source, **Work on · Edit · Add a source · Build** on a project
C	*Read the sources* read **one** source and said "sources"	**Read every source** reads them all, one at a time (a rate-limited host is not asked twice at once), with one progress notice and one report
D	*Add a source* opened the whole edit form	the add form is on the hub's **Sources** tab: repository + branch, or a folder, or an archive
E	a build ran only the output script	a build runs the repository's own **`docs:registry`** first when it has one — that is what writes the index, the headings and the stale verdict — then the output script, and says which of the two it ran
F	the AI answered about the open folder only, and said nothing about what it was answering	the panel is **about the project**: it names it, and when no editor is in front it falls back to the page the project remembers you were on

The Library's *Projects* row (`extensions/temco-workbench/src/views.ts`) is the list of what this application manages. Each project's row carries what a project is made of and what can be done with it — the four technical facts are on the row and its children, so nothing has to be opened to decide: version · owner · outputs; its sources with how many pages each holds and how it was read; how many pages, how many without an owner, how many artifacts and when the last one was built; and the operations — *Work on it · Open · Read sources · Build · Edit · Remove*…. The one being worked on is first and marked, because the panel is read top-down.

A page for a project that has been removed closes itself, and the projects page takes its place. The first version kept the tab and put a notice in it — which was wrong twice over. It cleared `document.body`, taking the tabs, the actions and every element the next render writes into with it, so the tab could never become another project's page; and it left a page *about nothing*, with working-looking buttons on it. A tab for something that no longer exists is worse than no tab: the removal says so in one message, closes that page, and opens the list of what is left. Removal from any surface does this, and a page opened for a project that is not there does the same thing rather than rendering.

There is no *Work* view. It was a second copy of the tree's *Gaps* list plus a counter, in a third place, behind one icon that meant five things at once — decided with the user. What replaced it:

- the sidebar's **Library** holds one expandable **Needs attention** row: a count line, the missing-file row, a stale row, an unowned row, then the faulty pages, then **More…**
- **More…** (or the row's title, or the row's title-bar button) opens **What needs doing** — a page with the room to say *why* each thing is listed and *where* the fix is: sources · faults · declared-but-missing · stale · no owner · owed. A missing file is created from there with one click, which is the one write this application is allowed
- each mark means **one** thing: `⊗` a structural fault, `!` a declared page whose file is absent, `↺` the generator's stale verdict, `▲` an unowned page, `⇄` owed
- the **staleness report** moved to the Library's title bar, where the view it belonged to used to be

Owed is never computed here. *Documentation owed* is the product repository's own gate — `scripts/docs-owed.mjs`, `npm run docs:owed`, waivers that expire (`11-review-and-cicd.md`) — so the page runs that script as a task and reports what a run is: where, when, the exit code. Its output, in the terminal, is the verdict, and the page says so. A second opinion about what is owed is the one thing a documentation platform must not invent.

The selection is state, in one place (`extensions/temco-store/src/selection.ts`, in `workspaceState`): which project, and which page of it you were last on. A project belongs to the window, not the machine — two windows on two products must not answer about each other's pages. Opening a project makes it the one being worked on, and the library row, the view's description and the dashboard all say so; a project that is removed while selected clears the selection rather than leaving a view about something that is not there.

On activation, nothing has to be clicked. The one project with nothing chosen selects itself (with two or more nothing is guessed — see *Surfaces*); the Workspace view arrives with the only repository's areas open and their chapters beneath them; the Library opens *Projects*, so the names of what you have are on screen; and the tab that opens is the welcome page (`temcoWorkbench.start.open`, default `welcome`), or — once that checkbox is off — the selected project's pages, then the page you were last on, then the overview. A tree that re-decided itself after every refresh would be worse than one that never opened at all, so a person's own expand or collapse is remembered for the session and beats both defaults — a refresh re-reads the data, never the layout (`extensions/temco-workbench/src/views.ts`).

The window

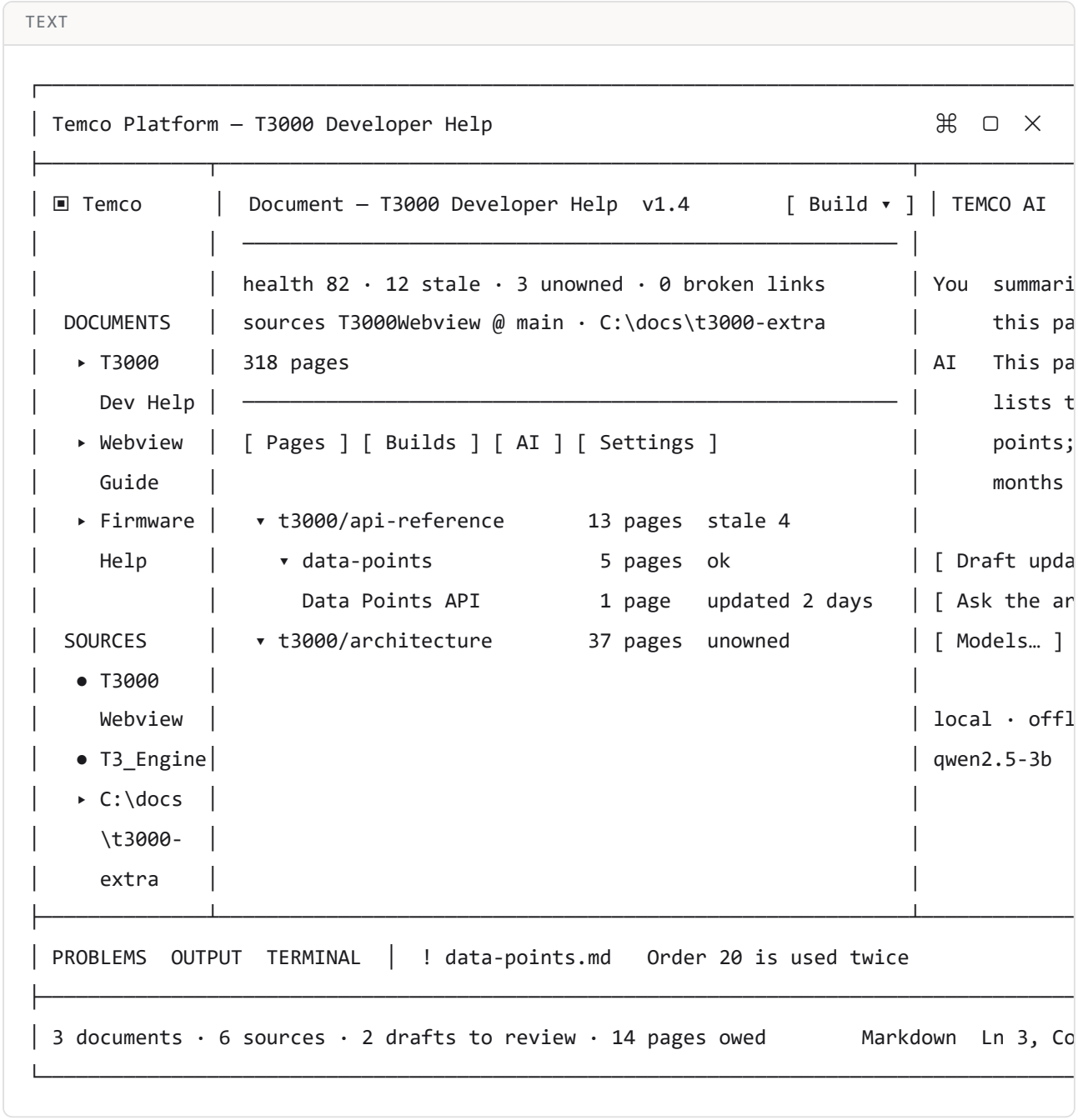
As built$2 The figures below are the design; what exists is these same surfaces with different names, and the difference is recorded here rather than left to be discovered:

- The sidebar holds **three** sections in one container, not two: **Workspace** (the page tree, the first thing built), **Work** (what needs doing — faults per open folder, and changes not yet shown), **Library** (the

projects, the sources with their pages beneath them, and the catalog's own state).

- The unit is called a **project** in everything a person reads. The catalog's table is still `documents`; a project is what that row is, and renaming the table buys nothing but a migration.
- **Four pages, one job each.** The **hub** (`extensions/temco-projects/src/projectHub.ts`) is what one project *is* — Overview · Pages · Sources · AI · Builds, with Read and Build at the top; the list of projects is the Library's *Projects* row, not a page of its own. The **edit page** (`extensions/temco-projects/src/projectPage.ts`) is what it is *made of*. Opening the editor for every look at a project would make reading its pages and editing its fields the same act, so they are separate commands (`openProject` and `editProject`). The **dashboard** (`extensions/temco-workbench/src/home.ts`) is the application itself, and the **AI page** (`extensions/temco-ai/src/aiPage.ts`) is the AI. All four are tabs in one group.
- **The project's pages are tabs in one group**, and so are the documentation pages they open — never a split. The rule lives in `extensions/temco-ui/src/columns.ts` and is stated there once, together with the two wrong answers that led to it: a page opened `Beside` produced a second column a few hundred pixels wide, and "is the tab in front one of ours?" — tried twice — is not a question a webview tab answers reliably. So the file asks nothing about what is in front; it finds the group that already holds a page of ours and opens there, which makes one group the only outcome. `reveal` with that column also *moves* a page a previous layout left in a second group, so the split heals the next time the page is opened. `ViewColumn` is named in exactly one place in the whole source.
- **The layout survives a narrow column.** Every grid is `minmax(min(Nem, 100%), 1fr)`, every flex child that holds text is `min-width: 0`, and a table wider than the pane scrolls inside its own box. A bare `minmax(Nem, 1fr)` sets a hard floor that pushed the right-hand column out of the pane — the page scrolled sideways instead of reflowing, and it looked like a rendering fault rather than a CSS one.
- The **AI page** (`extensions/temco-ai/src/aiPage.ts`) is the AI as something owned and configured — the four facts a person needs before trusting an answer, what can be downloaded, and which is chosen. It stands beside the AI *panel*: the panel is the conversation, the page is the setup. A project's AI tab carries the same facts plus the suggestions read from that project's pages.
- **Suggestions need no model.** The ones on the AI tab — an unowned page, a source never read, a chapter past about 25 pages, a repeated title — come from the catalog, so they are identical with the AI off. What a model adds is prose over the same facts, never the facts themselves.

TEXT



The AI goes on the right, beside the page being worked on — that is where `app-layout.md`'s mock puts it, and it is where a reader of a page wants a question answered.

The AI sits in the right side bar, and that is a declaration, not a preference. The container is contributed under **`contributes.viewsContainers.secondarySidebar`** — a location VS Code's own schema has, and which this design document wrongly claimed did not exist. The secondary side bar is therefore its default home: nothing to drag, nothing to pick, no setup step. If someone moves it (the panel, the primary side bar), *Show the AI in the right side bar* (`temcoAi.dockRight`, in the `temco-ai` module) focuses the view and invokes the per-container `<containerId>.resetViewContainerLocation` command VS Code registers, which returns it to its declared location. That is also why the extension never asks *where* a view currently is: the view-location API is not exposed to extensions, and the reset command makes the question unnecessary.

The toolbar

The design package puts the editor on the **website** — `15-online-editor.md`, *The editor core*, picks Monaco, *The screen* gives the toolbar it needs, and *The API* the contract behind it. Two of those three are already true on the desktop without a line of ours, and saying which is which is what keeps this small:

TEXT

```
what 15-online-editor.md asks for            the website has to      this application
_____________________________________________ ______________________ ________________

Monaco, the editing core                      bundle it, ~2 MB        VS Code *is* Mon
find, folding, breadcrumb outline, minimap    free once bundled       the same, from t
gate findings as squiggles on the line        /api/docs/validate      src/diagnostics.
a toolbar of formatting buttons               build it in main.js     src/format.ts —
link completion from index.json               a completion provider   Link to a page,
a WYSIWYG mode                                refused on principle    refused for the
preview identical to the site                 docfx serve in an iframe the repository'
publish, and the PR it opens                  POST /api/docs/publish  never: it builds
```

The one row that *is* work is the toolbar, and it lands in the editorial surfaces a person already uses rather than in a page of ours: four buttons in the editor's title bar for a Markdown file, the same sixteen actions in the submenu on the editor's context menu and on the tab's, and one **Format** entry that lists them all, so the palette is the keyboard path. `15-online-editor.md`, *The screen* asks for the same list, and the set matches it: emphasis, headings, the lists, the quote, the code block, the table, the rule, an image, a link to a page, an outline.

Three decisions inside it are worth recording, because each one had two answers:

Decision	Taken	Why
The editor is VS Code's, with the project's toolbar on it	**not an embedded editor page**	the design package refuses an embedded VS Code for the website (`15-online-editor.md`, *What is not built*); here it would be embedding the editor inside the editor, and re-building the preview, the markers and the file handling that already exist
No Markdown formatting commands ship in VS Code	**so these are ours**	checked, not assumed: no `toggleBold` exists in any built-in extension, so `**…**` had no button before this
No keybindings are contributed	**deliberate**	`Ctrl+B` hides the side bar everywhere else, and this extension must keep working in an ordinary VS Code. Every action is a command named `temcoDocs.format.*`, so a chord is one binding away for whoever wants it

The button that needed a rule rather than an implementation is **Link to a page**. `05-content-and-registry.md` fixes the form — a *relative* `.md` link, resolved and warned on by the build, with `~/` for root-relative — so the picker offers the pages the registry declares and inserts a path the gate can resolve. A page in another source is refused in words, because no relative path reaches it. **Callout** takes its five kinds from DocFX's own alerts (`> [!NOTE]`, `TIP`, `IMPORTANT`, `CAUTION`, `WARNING`), which is the syntax the renderer actually implements.

An image is the one button that is not here, and the reason is the write rule rather than the code: the file has to be written into the page's own `media/` before the reference to it points at anything. That amendment, the kinds of new document, and what each costs are the subject of `docs/editor-design.md`.

Operating on things

The operations were in the sidebar's title bar — twelve buttons on the Library, four of them needing a target nobody had picked — and the tree could only be read. What replaced it is one rule and one place per kind of thing:

```text
 the title bar of a view        starts work, and stands for nothing in particular
 the ⋯ in that title bar         everything that needs a target, or is housekeeping
 a row                          operates on the thing the row is
 a page (dashboard, hub)        operates on the thing the page is about
```

```text
LIBRARY HEADER — before

┌──────────────────────────────────────┐
│ [+][▤][✓][◧][+ᵥ][↓][↓][●][⚙][☰][⌘][▶] │   12 buttons — and four of them have no t
└──────────────────────────────────────┘


LIBRARY HEADER — after

┌──────────────────────────────────┬───┐
│ [+ New project] [Projects] [✓]│ ⋯ │   3 buttons, and the ⋯ holds everything else
└──────────────────────────────────┴───┘

  ⋯   Read every source · Read a source… · Add a source…
      Work on a project… · Build the project…
      Run a script… · Staleness report · Catalog · Set a token
```

The Workspace tree now carries the page operations, because that is where a page is. `page` and `missingPage` are two context values rather than one, so the menu can offer *Create the missing page file* only where a file is missing, and *Move* only where there is one.

TEXT

```
▾ T3000  t3000 · 318 pages
  ▾ features  41 pages  ⇄ owed
      Trend Logs      ● 2 issues
      Data Points     ▲ unowned
      System Time     ⚠ missing file
      Trend Log       (read from the source)

  left click a row        a page opens · a folder expands          (the Explorer's own
  right click a row       the row's menu — and the keyboard's menu key opens the same o
  on a folder row         New Page… · New Chapter… · New Page of a Kind ▸ · Reveal in E
                          Open in Integrated Terminal · Copy Path · Copy Relative Path
                          Open the Registry
  on a page row           Open the Page · Reveal in Explorer · Open in Integrated Termi
                          Copy Path · Copy Relative Path · Rename the Title…
                          Move the Page Up · Move the Page Down · Retire… · Open the Re
  the view's toolbar      [New page] [New chapter] [Refresh] […]  — the … is the list
                          row the tree last had focused
  drag a page onto        a chapter (chapter + order) · a sibling (two order values, sw
```

The tree behaves the way the Explorer does, because that is the tree everyone already knows. A left click opens a page or expands a folder; a right click opens the row's own menu, which carries only what that row can do — a page has no *New Page…*, a chapter has no *Rename*. Nothing is on hover, because a menu started from a row can only ever be a quick pick in the middle of the top of the window, and the menu above is what replaces it. The one act that is *not* the Explorer's is deliberate: **the view's `…` shows the same list for the row the tree last had focused**, so the buttons in the title bar and the palette reach every act without a right click (a view title action runs with the tree pane's context, and the row commands fall back to the view's selection).

The kinds are the file types. A page is always Markdown — its id is its path under `docs/`, the generator reads its title from its first `#` — so the submenu lists what actually differs between one new page and another: the headings it starts with (Article · Task · Reference · Troubleshooting · Release note).

A drag is a registry write, and it says which values it will change. Dropping a page on a chapter writes its `chapter` and a new `order`, and the confirmation says the word *structure*, because `15-online-editor.md`, *Safety rules*, rule 4 marks that field as the docs owner's. Dropping it on a sibling in the same chapter swaps two `order` values and nothing else. Nothing here renumbers a chapter: placing a page between two others is the generator's own job, and the generator does it for whatever nobody curated.

The projects page is the operations home, which it already was — each card carries *Work on it* · *Open* · *Read sources* · *Build* · *Edit* · *Scripts…* · *Remove…* — with one addition: **Scripts…** lists the repository's own allowlisted scripts that *this* project's folders actually have, and runs the one you pick through the

Library's own command. A script belongs to a repository, so it is offered per project only because a project is what cites one; when the project is not the one being worked on, it is made so first, out loud, because the Workspace view changes with it.

Documents — the home page

```text
DOCUMENTS                                              [ + Create a document ]   [ Scan

Since you last looked
  ● T3000 Developer Help      12 pages changed · built 2 days ago · 2 drafts waiting
  ● C:\docs\t3000-extra       new folder found · 34 pages · never built

DOCUMENT                VERSION  SOURCES  PAGES  HEALTH  LAST PUSH          STATE
T3000 Developer Help    1.4      2        318    82      main 2 days ago    ok · 12 st
  ├ 12 stale pages — the code they claim moved on since the last review
  └ 3 pages have no owner
T3000Webview Guide      2.0      1        54     94      main 3 weeks ago   ok
Firmware Help           0.9      1        96     61      rel/2.1 5 months   stale, 4 u

4 documents · 6 sources · 468 pages · 1 source has no registry yet

(nothing here yet)   No documents yet.  [ Create a document ]   [ Scan for sources ]
```

Sources — what the platform reads from

```text
KIND     SOURCE                    BRANCH      REGISTRY  DOCS   PAGES  LAST PUSH
git      T3000Webview              main        —         yes    318    main 2 days a
  └ registry is on feature/lvgl-svg-renderer, not on main          ⚠ branch mismatch
git      T3_Engine                 release/2.1 yes       yes     96    rel/2.1 5 mon
git      T3000-API                 main        —         —       —     main 3 years
folder   C:\docs\t3000-extra       —           —         yes     34    —
folder   C:\docs\legacy            —           yes       yes    104    —
archive  vendor\bacnet-stack.pdf   —           —         —       —     —
… 17 more repositories with no documentation found

[ Scan for sources ]  [ Add a source… ]  [ Prune the source cache (1.8 GB of 2 GB) ]
```

Create a document

TEXT

```
┌─ Create a document ──────────────────────────────────────────────────────────
│  1  Name        T3000 Developer Help    version 1.4    audience developers    owner
│
│  2  Sources — pick from what the catalog knows                        [ + Add a sou
│     [x] T3000Webview    branch [ main ▾ ]    318 pages · registry —        ⚠ see be
│     [x] C:\docs\t3000-extra                  34 pages · no registry
│     [ ] T3_Engine       branch [ rel/2.1 ▾ ]  96 pages · registry yes
│       the platform says what it found:
│         the registry is on feature/lvgl-svg-renderer, not main   → [ use that branch
│         C:\docs\t3000-extra has 34 pages and no registry         → [ start one ]
│
│  3  Recipe      engine [ docfx ▾ ]   config docs/docfx.json
│     outputs     [x] CHM t3000_dev.chm  [x] PDF t3000_dev.pdf  [ ] publish to the sit
│     it runs the repository's own script, as a task — never a re-implementation
│
│  4  Review      T3000 Developer Help v1.4 · 2 sources · 352 pages · chm + pdf
│                 first build:  npm run docs:registry  then  npm run docs:site
│                                                  [ Back ]  [ Create ]  [ Create &
└──────────────────────────────────────────────────────────────────────────────
```

Reading a source — three modes

TEXT

```
local      a folder you have open, or a path in the catalog
           files · checks · edits · builds                                  cost: no

fetch      a shallow clone into  globalStorage/sources/<slug>
           git clone --filter=blob:none --depth 1                           cost: di
           used for: drafts (the model needs the page text), builds, file-existence ch

api        one commit's changed files + patches, over the host's API
           used for: commit summaries, "what changed"                       cost: ca
           rule: summaries need diffs · drafts need files
```

AI analysis — the job, and what it produced

```text
AI — analysis over history                                  model qwen2.5-3b · local ·
──────────────────────────────────────────────────────────────────────────────────────

Job    18 of 24 commits summarised · 6 remaining      [ Run ]  [ Pause ]  [ Run the re
       T3000Webview @ main · since 2026-09-01 · skipped 1 (patch too large: 1.4 MB)

SUMMARISED
  411c5e55  2 days ago   docs   Clean the docs and move them into the platform reposit
                         → moved 3 chapters, rewrote the design hub intro
  f27cf000  5 days ago   docs   Update the docs to add a YAML file for DocFX
                         → affected t3000/architecture (2 pages)

DRAFTS TO REVIEW
  t3000/architecture/overview.md   +12 −4   "the registry now declares a YAML file…"
  t3000/data-points/README.md      +3  −0   "the Design Studio section moved"

PAGES OWED   14 pages · oldest 8 months      [ Open the owed list ]

Nothing here is published or committed: applying a draft is a diff you accept, one `Ct
```

Which page reads what

```text
Documents page   ← documents · pages · builds · artifacts · events
Sources page     ← sources · scans · pages · commits
Document page    ← doc_sources · recipes · builds · artifacts · pages · commits
Create a doc     ← sources (what exists)   → documents · doc_sources · recipes
AI analysis      ← commits · commit_pages · pages   → summaries · drafts · owed
Build log        ← builds · artifacts
Notifications    ← events where seen_at is null
```

What every surface obeys

- **Say where the facts came from.** Every row carries its fidelity — local, fetch or api · floor.
- **Empty states act.** A page with nothing on it offers the one thing that applies, never a sentence telling the reader to go and find it.
- **A problem lands where the fix is** — the page for a page problem, its row for a document that has never been built.

- **Nothing is written to a source**, and nothing is published, except by a command the user ran and confirmed.

Decisions

Decision	Made
The AI's dock	docked on the **right by the workbench** — the secondary side bar takes any view and remembers the position. No contribution point can place it there, so a fresh install needs one drag, and that drag is the documented setup step
The catalog's store	`node:sqlite` — built into the Node the extension host runs, so the platform adds no dependency for it. Where that built-in is missing, the store refuses in words at the point of use and activation is unaffected
Branch policy	the default branch, with the mismatch reported and a per-source override
Notification default	`docs-only` — a daily summary of documentation change, no modals
The entry point	**the welcome page opens on activation** (`temcoWorkbench.start.open` , default `welcome` , with the checkbox in its own footer as VS Code has it), falling through to the project's pages, then the page you were last on, then the overview. The one recorded project selects itself, so the tree is ready without anyone picking it
The Workspace, gated by content	**the Workspace stays the first view and the Library sits beneath it**, and the open folders are never read into it. Its empty states are **rows the tree draws itself**, offering the act that fills it — *create a project · choose one in the Library · open this project and read its sources*. Hiding the view on a context key (`when:`) and phrasing these states as `viewsWelcome` were both tried and rejected: a selected project with no visible tree reads as a fault, and a welcome's `when` is resolved before the refresh has read anything

Build order

Step	What	Done when
R1	**The catalog** — the store, its schema, migrations, and the rows above	**done**: 12 tables, applied idempotently and exercised against a real `node:sqlite` database — inserts, the document↔source join, the unseen-events query. The store lives in `globalStorageUri`, never in a repository
R2	**Sources** — add a git source (any host, chosen branch) or a folder; local, materialised and online modes	**partly built**: a source is added, read in `local` and `api` mode, and remembered across restarts. Not built: the materialised (`fetch`) mode, and the branch mismatch is reported by the Content view rather than by a read
R3	**Documents** — create one, cite sources, choose a recipe, and build it	**partly built**: a project is created, cites sources, chooses a recipe, and its build runs the repository's own script as a task and records the files by path. Not verified: a real artifact (`t3000_dev.chm`) has not been produced through this path yet
R4	**AI over history** — commit summaries, affected pages, drafts, the owed list, as a resumable job	a real commit has a summary and a draft, and a human still approves
R5	**The views** — Documents and Sources: health, fidelity, latest push, *Since you last looked*	**partly built**: the sidebar holds Workspace (the pages of the project being worked on, with a row saying why while there is nothing) first and Library (projects · sources · repositories · catalog · needs attention · the AI) beneath it, and *What needs doing* answers "what do I have, and what changed?" on a page. Not built: the *Since you last looked* digest, and fidelity per source beyond the read mode
R6	**Editing and publishing** — the editor with its toolbar, creating a document, images, draft branches, the design package's hosted editor, PR flow	**partly built**: the **toolbar** is built on VS Code's own editor — emphasis, blocks, a link to a real page, the outline — as the sixteen `temcoDocs.format.*` commands in `src/format.ts` . A page can be **created, named, moved and retired** — `src/pageOps.ts` , six commands and a drag controller — and a project's own scripts can be run from its card on the projects page. Not built: inserting an image, drafts and the PR flow, which need the `fetch` read mode and a review flow that lives in the product repository

R6 is mostly a rule change, and the rule was changed by naming it. The toolbar edits the buffer a person is already typing in, so it adds no writer at all. Creating a page writes **one file** — the repository's own `docs:registry` adds the entry, derives its place from the path and its title from the first heading, so nothing here writes an entry for a page that did not exist. And a page that is *moved* changes **one**

curated value at a time in `docs/help/registry.json`, through `src/registryEdit.ts`, which renders the file the way its own generator does — including the file's line endings — so a move is a two-line diff. `docs/editor-design.md` §5 records the decision, the four rules that hold it, and the evidence.

R1–R3 make it a documentation platform: sources in, documents and artifacts out. R4 makes it intelligent. R5 makes it a management system. R6 closes the loop.

What this does not change

- **Content never moves.** A document cites sources; it never relocates their pages.
- **Only artifacts ship.** A customer installation receives the CHM and the F1 map — never this app.
- **The AI proposes; a person approves.**
- **The design package still wins** on content, registry and publishing rules. This document only says what the platform knows, where it keeps it, and what it builds.